



沈阳建筑大学

上机实验报告

课程名称:

操作系统

实验题目:

存储器管理模拟实验

专业班级:

计算机 2101 班

学 号:

2104230414

日 期:

2024.05.04

【实验目的】

1. 通过编写和调试存储管理的模拟程序以加深对存储管理方案的理解；
2. 熟悉虚存管理的各种页面淘汰算法；
3. 通过编写和调试地址转换过程的模拟程序以加强对地址转换过程的了解。

【实验准备】

虚拟存储器的管理方式

- 段式管理
 - 页式管理
 - 段页式管理
2. 页面置换算法
 - 先进先出置换算法
 - 最近最久未使用置换算法
 - Clock 置换算法
 - 其他置换算法

【实验内容】

1. 实验题目

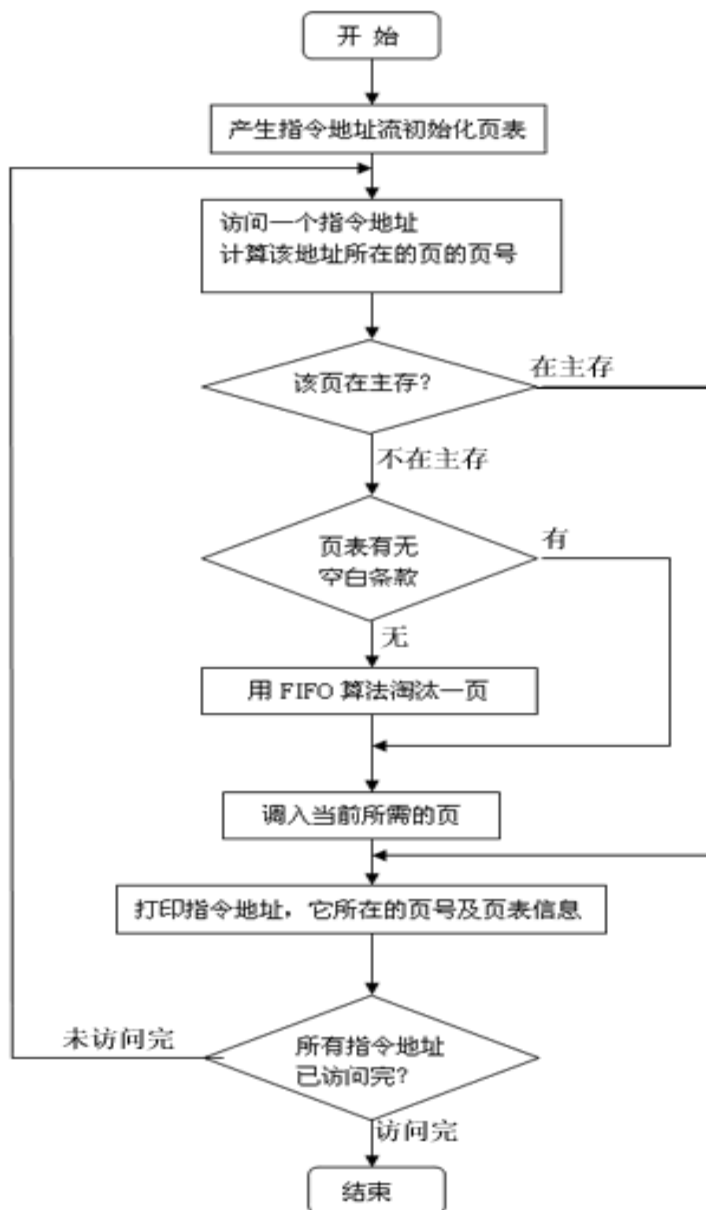
设计一个请求页式存储管理方案。并编写模拟程序实现之。产生一个需要访问的指令地址流。它是一系列需要访问的指令的地址。为不失一般性，你可以适当地（用人工指定地方法或用随机数产生器）生成这个序列，使得 50% 的指令是顺序执行的。25% 的指令均匀地散布在前地址部分，25% 的地址是均匀地散布在后地址部分。为简单起见。页面淘汰算法采用 FIFO 页面淘汰算法，并且在淘汰一页时，只将该页在页表中抹去。而不再判断它是否被改写过，也不将它写回到辅存。

2. 具体做法

产生一个需要访问的指令地址流；指令合适的页面尺寸（例如以 1K 或 2K 为 1 页）；指定内存页表的最大长度，并对页表进行初始化；每访问一个地址时，首先要计算该地址所在的页的页号，然

后查页表，判断该页是否在主存——如果该页已在主存，则打印页表情况；如果该页不在主存且页表未满，则调入一页并打印页表情况；如果该页不在主存且页表已满，则按 FIFO 页面淘汰算法淘汰一页后调入所需的页，打印页表情况；逐个地址访问，直到所有地址访问完毕。

3. 存储管理算法的流程图



【源程序代码】

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <iomanip>
#include <string>
#include <ctime>
#include <windows.h>
#include <fstream>

using namespace std;
//以下是页面信息类的定义
class PageInformation
{
protected:
    int pid;           //页号
    int count;         //统计驻留在物理块中的次数（时间）,初始为 0
public:
    PageInformation() { count = 0; }           //默认构造函数，驻留在页面的
    时间初始化为 0
    PageInformation(PageInformation& p)
    {
        p.pid = pid;
        p.count = count;
    }
    virtual ~PageInformation() {}
    void setpid(PageInformation* headptrblock, int pid)//修改物理块对象数组成员
pid
    {
        headptrblock->pid = pid;
    }
    void setpidrand(PageInformation* headptr, int pid)//修改页面对象数组成员 pid
    {
        headptr->pid = pid; cout << pid << " ";
    }
    void display()
    {
        cout << pid << " ";
        //cout<<pid;
    }
}
```

```

void setpid()//选择从键盘输入时设置对象数组成员 pid
{
    int pidtemp;
    cout << "请输入您的页号（范围 1-9）： ";
    cin >> pidtemp;
    this->pid = pidtemp;
}
void counter()
{
    count++;
}
void resetcount()
{
    count = 0;
}
int getpid()
{
    return pid;
}
int getcount()
{
    return count;
}
};//PageInformation

void Directloading(PageInformation*& headptr, PageInformation*& headptrblock, int
pagenum)//直接将前三个页面装入物理块中
{
    int temp;
    for (int i = 0; i < pagenum; i++)//直接将符合物理块数大小的页面装入物理块
中
    {
        (headptrblock + i)->setpid((headptrblock + i), (headptr + i)->getpid());
        temp = i + 1;
        for (int j = 0; j < temp; j++)
            (headptrblock + j)->counter();//对应物理块驻留时间++
    }
    cout << "原始页号为： " << "";
    for (int i = 0; i < pagenum; i++)

```

```

{
    (headptrblock + i)->display();
}
cout << endl;
} //Directloading

void FIFO(PageInformation* headptr, PageInformation* headptrblock, int pagenum,
int pagenumber, int& pagecounter)
{
    Directloading(headptr, headptrblock, pagenum);
    pagecounter = 0; //置 0
    int status = 1;
    int temp = 0;
    for (int i = pagenum; i < pagenumber; i++) //如果有缺页现象发生，就将存入物
理块时间最久的一个页面取出，装入新的页面
    {
        for (int j = 0; j < pagenum; j++)
        {
            (headptrblock + j)->counter(); //对应物理块驻留时间++
        }
        for (int j = 0; j < pagenum; j++) //无缺页跳出循环进行下一页面判断，缺页
寻找驻留时间最长的页面置换之
        {
            if ((headptrblock + j)->getpid() == (headptr + i)->getpid())
            {
                status = 0;
                pagecounter++;
            }
        }
        for (int j = 0; j < pagenum; j++)
        {
            if ((headptrblock + j)->getcount() > (headptrblock + temp)->getcount())
                temp = j;
        }
        if (status)
        {
            (headptrblock + temp)->setpid((headptrblock + temp), (headptr +
i)->getpid()); //将新页面装入物理块置换掉 temp 页面

```

```

        (headptrblock + temp)->resetcount();
    }
    cout << "第" << setw(2) << i + 1 << "页所持有的页号为: ";
    for (int j = 0; j < pagenum; j++)
    {
        (headptrblock + j)->display();
    }
    if (status)
    {
        cout << "未命中! ";
    }
    cout << endl;
    //将临时暂存变量更新
    temp = 0;
    status = 1;
}
int pagecounterrate = ((17 - pagecounter) * 100 / 20);
cout << "使用 FIFO 算法的缺页数为: " << 17 - pagecounter << "\n 缺页率为: " << "%" << pagecounterrate << endl;
} //fifo

int pagenum = 3;                //页框数-物理块数
int pagenumber = 20;            //页面数量
int pagecounter = 0;

//以下是主函数
int main(int argc, char** argv)
{
    time_t Time;
    time(&Time);
    int control;
    PageInformation* headptr = new PageInformation[pagenumber];    //生成页面
    序列
    PageInformation* headptrblock = new PageInformation[pagenum];    //生成页框
    if (argc == 1) //是否向 main 函数传入参数
    {
        cout << "随机生成的页地址流为: " << endl;
        srand((unsigned)time(&Time)); //srand((unsigned) time (NULL));也可以传
        入空指针

```

```

        for (int i = 0; i < pagenum; i++)
        {
            (headptr + i)->setpidrand((headptr + i), (int)rand() % 10);
        }
        cout << endl;

    }
    else//从文件中读入数据
    {
        cout << "(文件读入数据)" << endl;
        ifstream in(argv[1]);
        if (in.is_open())
        {
            cout << "文件打开成功！ " << endl;
        }
        else
        {
            cout << "文件打开失败！ ";
            exit(0);
        }
        cout << "所读取的页面序列为： ";
        int j = 0;
        for (int p; in >> p; )
        {
            (headptr + j)->setpidrand((headptr + j), p);//将文件数据读入
            j++;
        }
        in.close();
        cout << endl;
    }

    cout << "输入 2 使用 FIFO 替换算法进行替换： ";
    cin >> control;
    system("cls");
    cout << "（FIFO 先进先出置换算法） " << endl;
    FIFO(headptr, headptrblock, pagenum, pagenum, pagecounter);
    delete[] headptr;
    return 0;
}

```


运行结果截图：

```
E:\操作系统实验3.存储器管理 x + ~ - □ x
(FIFO先进先出置换算法)
原始页号为: 6 6 0
第 4页所持有的页号为: 5 6 0 未命中!
第 5页所持有的页号为: 5 3 0 未命中!
第 6页所持有的页号为: 5 3 0 未命中!
第 7页所持有的页号为: 5 3 8 未命中!
第 8页所持有的页号为: 6 3 8 未命中!
第 9页所持有的页号为: 6 5 8 未命中!
第10页所持有的页号为: 6 5 7 未命中!
第11页所持有的页号为: 4 5 7 未命中!
第12页所持有的页号为: 4 9 7 未命中!
第13页所持有的页号为: 4 9 3 未命中!
第14页所持有的页号为: 4 9 3
第15页所持有的页号为: 4 9 3
第16页所持有的页号为: 4 9 3
第17页所持有的页号为: 5 9 3 未命中!
第18页所持有的页号为: 5 9 3
第19页所持有的页号为: 5 9 3
第20页所持有的页号为: 5 0 3 未命中!
使用FIFO算法的缺页数: 11
缺页率为: %55

-----
Process exited after 2.269 seconds with return value 0
请按任意键继续. . .
```

【实验体会】

这次是第三次操作系统实验，通过对存储器的管理进行模拟，更加深入了解了各种的页面置换算法：FIFO、LUR、OPT，提升了对存储器管理机制的理解和操作能力，也提升了我的代码操作水平。

【思考题】

几种页面置换算法各有什么优缺点？

1. 先进先出（FIFO）页面置换算法

- 优点：实现简单，易于理解。
- 缺点：可能产生 Belady 异常（即增加缺页率的现象），不适用于实际的工作集模型，可能导致最近最少使用的页面被替换掉。

2. 最近最少使用（LRU）页面置换算法

- 优点：根据页面的使用历史来替换最长时间未使用的页面，较好地反映了程序的局部性原理。
- 缺点：实现复杂，需要维护一个记录所有页面访问历史的队列，可能导致较高的硬件成本和计算开销。

3. 最优（Optimal）页面置换算法

- 优点：理论上的最佳页面置换策略，替换未来最长时间不再被访问的页面。
- 缺点：实际上无法实现，因为需要知道未来的访问模式。